



دانشگاه علم و صنعت ایران

دانشکده مهندسی کامپیوتر

طراحی، پیاده‌سازی و ارزیابی واحد واکشی دستور (IF) و رجیستر IF/ID در پردازنده Pipeline با استفاده از VHDL

گزارش آزمایش هشتم (پروژه نهایی)
درس آزمایشگاه معماری کامپیوتر

تهیه کننده:

آرین سعیدکندری ۴۰۲۵۲۱۳۰۶

استاد درس:

دکتر رضا غلامحسینی

بهار ۱۴۰۵

۱. مقدمه و هدف آزمایش

در این آزمایش مرحله واکنشی دستور (Instruction Fetch - IF) که اولین مرحله در یک پردازنده Pipeline است، طراحی و پیاده‌سازی شد. پردازنده‌های مدرن برای افزایش کارایی از تکنیک Pipeline استفاده می‌کنند که در آن مراحل مختلف اجرای دستورات به صورت همزمان روی چندین دستور انجام می‌شود.

واحد IF وظیفه دارد در هر سیکل کلاک دستور جدیدی را از حافظه واکنشی کرده و آماده ارسال به مرحله Decode نماید. هرگونه خطا در این مرحله می‌تواند عملکرد کل Pipeline را مختل کند.

اهداف اصلی:

- طراحی Program Counter (PC) به عنوان رجیستر ۸ بیتی با پشتیبانی از Freeze و Reset
- پیاده‌سازی حافظه دستورالعمل (Instruction Memory) به صورت ROM آسنکرون
- طراحی مالتی پلکسر انتخاب آدرس بعدی با پشتیبانی از Branch
- طراحی رجیستر IF/ID با پشتیبانی از Freeze و Flush
- اتصال تمام بلوک‌ها در یک Top Module و ارزیابی با Testbench
- سنتز و بررسی RTL Schematic و Resource Utilization در Vivado

۲. تئوری آزمایش

Pipeline چیست؟

پردازنده Pipeline دستورات را در مراحل موازی اجرا می‌کند. یک پردازنده استاندارد پنج مرحله دارد:

مرحله	نام
IF	Instruction Fetch
ID	Instruction Decode
EX	Execute
MEM	Memory Access
WB	Write Back

در هر سیکل کلاک، مرحله IF وظیفه تأمین دستور جدید برای Pipeline را بر عهده دارد.

ساختار واحد IF

واحد IF از چهار بخش اصلی تشکیل شده است:

الف) Program Counter (PC)

رجیستر ۸ بیتی که آدرس دستور جاری را نگهداری می‌کند. منطق به‌روزرسانی:

- حالت عادی: $PC_Next = PC + ۱$
- هنگام Reset: $PC \leftarrow ۰$
- هنگام Freeze: PC ثابت می‌ماند (Pipeline متوقف می‌شود)
- در پردازنده‌های واقعی مانند MIPS: $PC_Next = PC + ۴$ (هر دستور ۴ بایت)

ب) حافظه دستورالعمل (Instruction Memory)

یک ROM آسنکرون با مشخصات زیر:

ویژگی	مقدار
عمق حافظه	۱۶ خانه
عرض هر خانه	۳۲ بیت
نوع خواندن	Asynchronous (ترکیبی)
آدرس‌دهی	$PC[۳:۰]$

محتوای حافظه:

آدرس	محتوا
۰	۰x۰۱۲۳۴۵۶۷
۱	۰x۸۹ABCDEF
۲	۰x۱۱۲۲۳۳۴۴
۳	۰x۵۵۶۶۷۷۸۸
۴	۰x۹۹AABBCC
۵-۱۵	۰x۰۰۰۰۰۰۰۰

ج) مالتی‌پلکسر انتخاب آدرس (Next PC MUX)

یک مالتی‌پلکسر ۲ به ۱ که بر اساس سیگنال Branch_Taken آدرس بعدی را انتخاب می‌کند:

- اگر $Branch_Taken = 0$: $Next_PC = PC + 1$ (ادامه ترتیبی)
- اگر $Branch_Taken = 1$: $Next_PC = Branch_Address$ (پرش به آدرس هدف)

د) رجیستر IF/ID

رجیستر ۳۲ بیتی بین مراحل IF و ID که:

- در لبه بالارونده کلاک دستور واکنشی شده را ذخیره می‌کند
- در صورت Freeze: مقدار قبلی را حفظ می‌کند
- در صورت Flush: مقدار $0x00000000$ NOP بارگذاری می‌کند
- در صورت Reset: مقدار NOP بارگذاری می‌کند

مفاهیم کلیدی

Hazard

تداخلاتی که مانع اجرای صحیح دستورات در Pipeline می‌شوند. سه نوع اصلی:

- Data Hazard: وابستگی داده‌ای بین دستورات
- Control Hazard: ناشی از دستورات Branch یا Jump
- Structural Hazard: استفاده همزمان چند مرحله از یک منبع مشترک

Freeze (Stall)

برای رفع برخی Hazard ها لازم است Pipeline متوقف شود. هنگام $Freeze=1$ هم PC و هم رجیستر IF/ID باید ثابت بمانند تا دستور جدیدی وارد Pipeline نشود.

Flush

پس از وقوع Branch ممکن است چند دستور اشتباه وارد Pipeline شده باشند. Flush آنها را با NOP جایگزین می‌کند.

۳. کد VHDL مدارها

۳.۱. Program Counter (pc_reg.vhd)

رجیستر ۸ بیتی با اولویت‌بندی: $\text{rst} < \text{freeze} < \text{PC_Next}$

```
pc_reg.vhd

C:/Users/radin/Downloads/arian/HW8_IF_Pipeline/HW8_IF_Pipeline.srscs/sources_1/imports/arian/pc_reg.vhd

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity pc_reg is
6      port (
7          clk      : in  std_logic;
8          rst       : in  std_logic;
9          freeze    : in  std_logic;
10         next_pc   : in  std_logic_vector(7 downto 0);
11         pc        : out std_logic_vector(7 downto 0)
12     );
13 end entity;
14
15 architecture rtl of pc_reg is
16     signal pc_r : std_logic_vector(7 downto 0) := (others => '0');
17 begin
18     process(clk)
19     begin
20         if rising_edge(clk) then
21             if rst = '1' then
22                 pc_r <= (others => '0');
23             elsif freeze = '0' then
24                 pc_r <= next_pc;
25             end if;
26         end if;
27     end process;
28
29     pc <= pc_r;
30 end architecture;
```

شکل ۱ - کد VHDL ماژول Program Counter (pc_reg.vhd)

توضیحات کد:

- ورودی‌ها: clk، rst، freeze، next_pc (۸ بیت)
- خروجی: pc (۸ بیت)

- سیگنال داخلی pc_r با مقدار اولیه صفر
- Process حساس به rising_edge(clk)
- اگر rst=۱ : pc_r ← ۰x۰۰
- اگر rst=۰ و freeze=۰ : pc_r ← next_pc
- اگر rst=۰ و pc_r : freeze=۱ بدون تغییر (ثابت)

۳.۲. مالتی‌پلکسر انتخاب آدرس (next_pc_mux.vhd)

منطق ترکیبی ساده برای انتخاب آدرس بعدی بر اساس branch_taken.

```
next_pc_mux.vhd

C:/Users/radin/Downloads/arian/HW8_IF_Pipeline/HW8_IF_Pipeline.srcs/sources_1/imports/arian/next_pc_mux.vhd

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity next_pc_mux is
5      port (
6          pc_plus1      : in  std_logic_vector(7 downto 0);
7          branch_address: in  std_logic_vector(7 downto 0);
8          branch_taken  : in  std_logic;
9          next_pc       : out std_logic_vector(7 downto 0)
10     );
11 end entity;
12
13 architecture rtl of next_pc_mux is
14 begin
15     next_pc <= branch_address when branch_taken = '1' else pc_plus1;
16 end architecture;
```

شکل ۲ - کد VHDL مالتی‌پلکسر انتخاب آدرس بعدی (next_pc_mux.vhd)

توضیحات کد:

- ورودی‌ها: pc_plus1 (۸ بیت)، branch_address (۸ بیت)، branch_taken (۱ بیت)
- خروجی: next_pc (۸ بیت)
- اگر branch_taken = '۱' : next_pc ← branch_address
- اگر branch_taken = '۰' : next_pc ← pc_plus1

- پیاده‌سازی به صورت یک concurrent signal assignment با when/else

۳.۳. حافظه دستورالعمل (instr_mem.vhd)

ROM آسنکرون ۱۶ خانه‌ای ۳۲ بیتی که به صورت یک آرایه constant پیاده‌سازی شده است.

```
instr_mem.vhd

C:/Users/radin/Downloads/Proj_CA_Lab8/HW8/HW8.srcs/sources_1/imports/Proj_CA_Lab8/instr_mem.vhd

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity instr_mem is
6      port (
7          addr      : in  std_logic_vector(3 downto 0);
8          instruction: out std_logic_vector(31 downto 0)
9      );
10 end entity;
11
12 architecture rtl of instr_mem is
13     type rom_t is array (0 to 15) of std_logic_vector(31 downto 0);
14
15     constant ROM : rom_t := (
16         0 => x"01234567",
17         1 => x"89ABCDEF",
18         2 => x"11223344",
19         3 => x"55667788",
20         4 => x"99AABBCC",
21         others => x"00000000"
22     );
23
24     -- This attribute prevents Vivado from optimizing away the ROM block
25     attribute rom_style : string;
26     attribute rom_style of ROM : constant is "distributed";
27
28     begin
29         -- Read inside a process so the ROM appears as a separate block in RTL view
30         process(addr)
31         begin
32             instruction <= ROM(to_integer(unsigned(addr)));
33         end process;
34
35     end architecture;
```

شکل ۳ - کد VHDL حافظه دستورالعمل (instr_mem.vhd) ROM

توضیحات کد:

- ورودی addr: (4 بیت، یعنی PC[3:0])
- خروجی: instruction (32 بیت)

- نوع داده: آرایه‌ای از ۰ تا ۱۵ از مقادیر ۳۲ بیتی
- مقداردهی اولیه با constant ROM شامل ۵ دستور نمونه + مقدار صفر برای بقیه خانه‌ها
- استفاده از attribute rom_style = "distributed" برای جلوگیری از بهینه‌سازی و حذف ROM توسط سنتزگر Vivado
- خواندن داخل یک process حساس به addr انجام می‌شود (نه به‌صورت concurrent signal assignment) تا ROM در RTL Schematic به‌عنوان یک بلاک مجزا و قابل‌شناسایی نمایش داده شود
- با این حال دسترسی همچنان از نظر رفتاری ترکیبی (آسنکرون) است: تغییر addr بلافاصله instruction را تغییر می‌دهد، بدون وابستگی به لبه کلاک

۳.۴. رجیستر IF/ID (if_id_reg.vhd)

رجیستر ۳۲ بیتی با اولویت‌بندی: rst < flush < freeze < بارگذاری instruction

```
if_id_reg.vhd
C:/Users/radin/Downloads/arian/HW8_IF_Pipeline/HW8_IF_Pipeline.srcs/sources_1/imports/arian/if_id_reg.vhd

1 library ieee;
2 use ieee.std_logic_1164.all;
3
4 entity if_id_reg is
5     port (
6         clk          : in  std_logic;
7         rst          : in  std_logic;
8         freeze       : in  std_logic;
9         flush        : in  std_logic;
10        instruction_in : in  std_logic_vector(31 downto 0);
11        instruction_out: out std_logic_vector(31 downto 0)
12    );
13 end entity;
14
15 architecture rtl of if_id_reg is
16     signal ifid_r : std_logic_vector(31 downto 0) := x"00000000";
17     constant NOP  : std_logic_vector(31 downto 0) := x"00000000";
18 begin
19     process(clk)
20     begin
21         if rising_edge(clk) then
22             if rst = '1' then
23                 ifid_r <= NOP;
24             elsif flush = '1' then
25                 ifid_r <= NOP;
26             elsif freeze = '0' then
27                 ifid_r <= instruction_in;
28             end if;
29         end if;
30     end process;
31
32     instruction_out <= ifid_r;
33 end architecture;
```

شکل ۴ - کد VHDL رجیستر IF/ID (if_id_reg.vhd)

توضیحات کد:

- ورودی‌ها: clk, rst, freeze, flush, instruction_in (۳۲ بیت)
- خروجی: instruction_out (۳۲ بیت)
- سیگنال داخلی ifid_r با مقدار اولیه NOP
- اولویت اول rst=۱: بارگذاری NOP (۰x۰۰۰۰۰۰۰۰)
- اولویت دوم flush=۱: بارگذاری NOP (پاک‌سازی دستور اشتباه پس از Branch)
- اولویت سوم freeze=۰: بارگذاری instruction_in (دستور جدید)
- در غیر این صورت (freeze=۱): ifid_r بدون تغییر (توقف Pipeline)

۳.۵. مازول نهایی (if_stage_top.vhd)

Top Module که تمام زیرماژول‌ها را به یکدیگر متصل می‌کند.

The image shows three side-by-side screenshots of the Verilog code for the `if_stage_top` module. The code is written in VHDL/Verilog and defines the top-level architecture of the IF stage. It includes port declarations for `clk`, `rst`, `freeze`, `flush`, `instruction_in`, and `instruction_out`. It also defines internal signals like `pc_plus1_unsigned`, `pc_plus1_s`, `branch_taken`, `branch_address`, `next_pc_s`, `next_pc_mux`, `pc_reg`, `instr_mem`, `ifid_instr_s`, and `ifid_reg`. The code instantiates components like `U_MUX` (Branch MUX), `U_PC` (PC Register), `U_ROM` (Instruction Memory), and `U_IFID` (IF/ID Pipeline Register). The architecture is implemented using a `begin` block with various assignments and component calls.

شکل ۵ - کد VHDL مازول نهایی (if_stage_top.vhd) IF Stage

توضیحات کد:

- سیگنال‌های داخلی: pc_s (8 بیت), next_pc_s (8 بیت), pc_plus1_s (8 بیت), instruction_s (32 بیت), ifid_instr_s (32 بیت)

- محاسبه PC+1 با استفاده از دو سیگنال میانی pc_unsigned و pc_plus1_unsigned انجام می‌شود تا این جمع‌کننده در RTL Schematic به صورت واضح‌تری قابل ردیابی باشد
- U_PC : نمونه‌سازی pc_reg با ورودی‌های clk، rst، freeze، next_pc_s
- U_MUX : نمونه‌سازی next_pc_mux با ورودی‌های pc_plus1_s، branch_address، branch_taken
- U_ROM : نمونه‌سازی instr_mem با آدرس pc_s(3 downto 0)
- U_IFID : نمونه‌سازی if_id_reg با ورودی‌های clk، rst، freeze، flush، instruction_s

۴. توضیح (if_stage_tb.vhd) Testbench

برای ارزیابی عملکرد واحد IF یک Testbench جامع طراحی شد که تمام سناریوهای خواسته‌شده در صورت آزمایش را پوشش می‌دهد.

سناریوهای تست:

سناریو	زمان	توضیح
Reset	ns ۲۵-۰	rst=۱ اعمال می‌شود تا PC و IF/ID به صفر برگردند
اجرای عادی	ns ۷۵-۲۵	rst=۰، PC به صورت ترتیبی از ۰x۰۰ افزایش می‌یابد
Freeze فعال	ns ۱۰۵-۷۵	freeze=۱ اعمال می‌شود، (۳ سیکل)، PC و IF/ID ثابت می‌مانند
Freeze غیرفعال	ns ۱۲۵-۱۰۵	Pipeline، freeze=۰ از ادامه می‌دهد (۲ سیکل)
Branch اول	ns ۱۳۵-۱۲۵	branch_taken=۱، branch_address=۰x۰۲، PC پرش می‌کند
Flush	ns ۲۱۵-۲۰۵	flush=۱ یک سیکل اعمال می‌شود، IF/ID = NOP می‌شود
Branch دوم + Flush همزمان	ns ۲۷۵-۲۵۵	branch_address=0x04 و بلافاصله flush=1، سناریوی واقعی حذف دستور اشتباه پس از Branch
Reset دوم	ns ۳۴۵-۳۲۵	rst=1 دوباره اعمال می‌شود تا عملکرد صحیح Reset پس از اجرای عادی تأیید شود

```

if_stage_tb.vhd
C:/Users/radin/Downloads/Proj_CA_Lab8/HW8/HW8.srcs/sim_1/imports/Proj_CA_Lab8/if_stage_tb.vhd

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity if_stage_tb is
5  end entity;
6
7  architecture tb of if_stage_tb is
8
9      signal clk          : std_logic := '0';
10     signal rst          : std_logic := '0';
11     signal freeze       : std_logic := '0';
12     signal flush        : std_logic := '0';
13
14     signal branch_taken  : std_logic := '0';
15     signal branch_address : std_logic_vector(7 downto 0) := (others => '0');
16
17     signal pc            : std_logic_vector(7 downto 0);
18     signal next_pc      : std_logic_vector(7 downto 0);
19     signal instruction   : std_logic_vector(31 downto 0);
20     signal ifid_instruction : std_logic_vector(31 downto 0);
21
22     constant CLK_PERIOD : time := 10 ns;
23
24     begin
25
26         DUT : entity work.if_stage_top
27             port map (
28                 clk          => clk,
29                 rst          => rst,
30                 freeze       => freeze,
31                 flush        => flush,
32                 branch_taken => branch_taken,
33                 branch_address => branch_address,
34                 pc           => pc,
35                 next_pc      => next_pc,
36
37                 pc          => pc,
38                 next_pc     => next_pc,
39                 instruction  => instruction,
40                 ifid_instruction => ifid_instruction
41             );
42
43         -- 10ns clock generation
44         clk_process : process
45         begin
46             while true loop
47                 clk <= '0'; wait for CLK_PERIOD/2;
48                 clk <= '1'; wait for CLK_PERIOD/2;
49             end loop;
50         end process;
51
52         stimulus : process
53         begin
54             -----
55             -- 1) Reset: PC and IF/ID register should clear to zero
56             rst <= '1';
57             wait for 25 ns; -- 2.5 clock cycles
58             rst <= '0';
59
60             -----
61             -- 2) Normal execution: PC increments each cycle
62             -- Instruction and ifid_instruction show ROM contents
63             wait for 50 ns; -- 5 normal cycles (PC: 0,1,2,3,4)
64
65             -----
66             -- 3) Freeze: PC and IF/ID must hold their values
67             -- PC is around address 5 so instruction is meaningful
68
69             freeze <= '1';
70             wait for 30 ns; -- 3 freeze cycles
71
72             freeze <= '0';
73             wait for 20 ns; -- 2 cycles after freeze released
74
75             -----
76             -- 4) Branch: jump from current PC to address 0x02
77             -- Address 2 holds instruction 11223344 (clearly visible)
78             -- PC at this point is around 0x06, so jump-back is obvious
79
80             branch_address <= x"02";
81             branch_taken <= '1';
82             wait for 10 ns; -- one full cycle with branch active
83             branch_taken <= '0';
84             branch_address <= x"00";
85
86             -- Several cycles after branch so the new PC sequence is visible
87             wait for 50 ns;
88
89             -----
90             -- 5) Flush: IF/ID holds a valid instruction (non-zero)
91             -- PC is near address 2 so instruction = 11223344
92             -- After flush, IF/ID must become 00000000 (NOP)
93
94             wait for 20 ns; -- let IF/ID fill with a valid instruction
95
96             flush <= '1';
97             wait for 10 ns; -- one flush cycle
98             flush <= '0';
99
100            -- Several cycles after flush so the NOP is clearly visible
101            wait for 40 ns;
102
103            -----
104            -- 6) Branch + Flush together: real pipeline hazard scenario
105            -- Branch to address 0x04 (instruction 99AABBCC)
106            -- Immediately flush to discard the wrong instruction in IF/ID
107
108            branch_address <= x"04";
109            branch_taken <= '1';
110            wait for 10 ns;
111            branch_taken <= '0';
112            branch_address <= x"00";
113
114            flush <= '1';
115            wait for 10 ns;
116            flush <= '0';
117
118            wait for 50 ns;
119
120            -----
121            -- 7) Second Reset: verify reset works after normal operation
122
123            rst <= '1';
124            wait for 20 ns;
125            rst <= '0';
126
127            wait for 30 ns;
128
129            wait;
130        end process;
131
132    end architecture;
    
```

شکل ۶ - کد VHDL فایل (if_stage_tb.vhd) Testbench

ویژگی‌های Testbench:

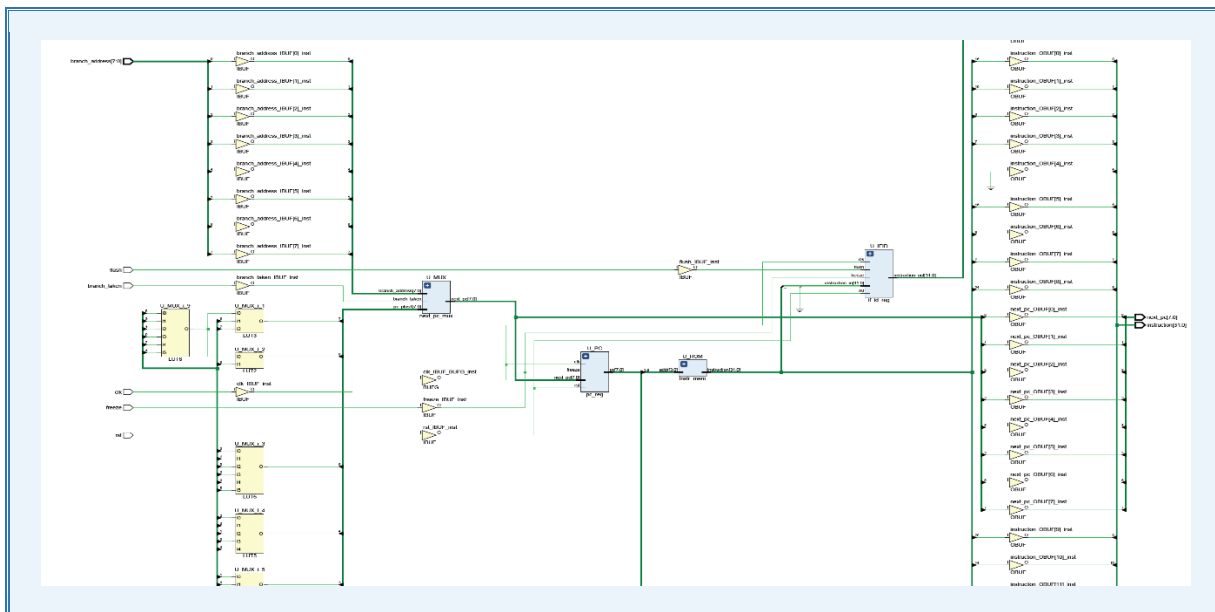
- کلاک ۱۰ ns با duty cycle 50% (CLK_PERIOD = 10 ns)
- تمام سیگنال‌های DUT با مقادیر اولیه امن مقداردهی شده‌اند

تحلیل Device View:

در نمای Device مشاهده می‌شود طراحی با موفقیت روی FPGA نگاشت شده است. چون مدار طراحی شده نسبتاً کوچک است، تنها بخش بسیار کوچکی از منابع فیزیکی FPGA اشغال شده و اکثر ناحیه‌های FPGA خالی هستند.

۶. تحلیل RTL Schematic

پس از سنتز، نمای RTL Schematic ساختار داخلی مدار را نشان می‌دهد.



شکل ۹ - RTL Schematic واحد IF Stage (شکل کامل بصورت pdf در فایل پروژه موجود است).

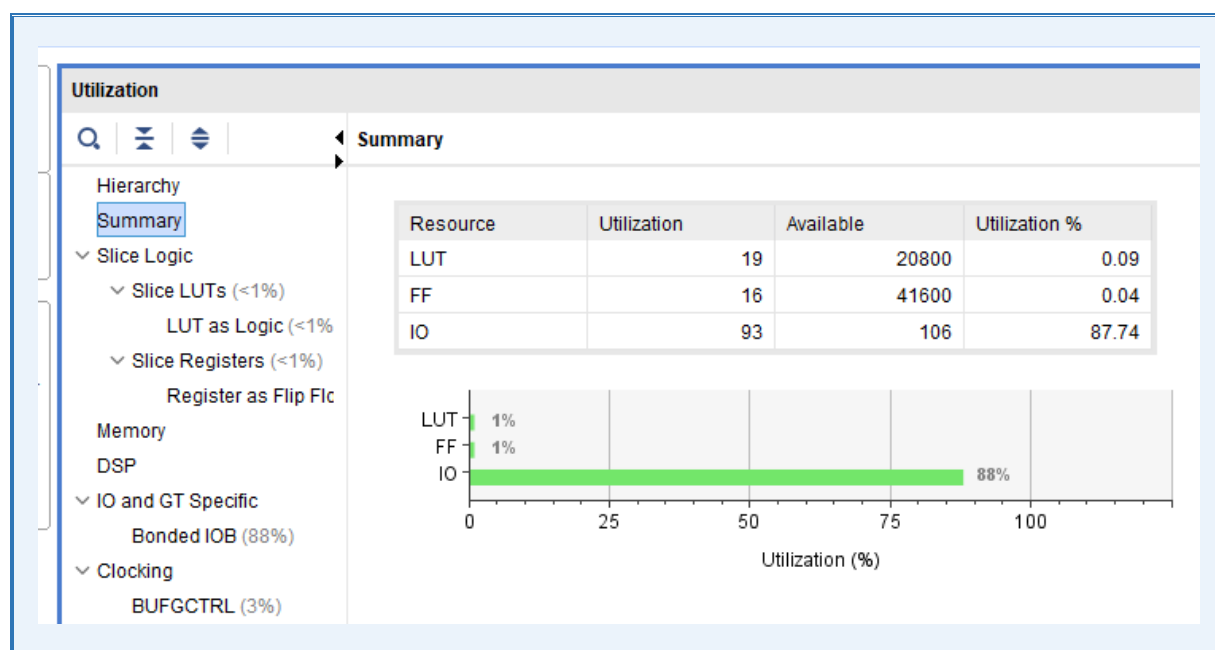
اجزای قابل مشاهده در RTL Schematic:

توضیح	نوع	بلوک
رجیستر ۸ بیتی PC با enable و reset	Sequential	U_PC (pc_reg)
مالتی‌پلکسر ۲ به ۱ انتخاب آدرس	Combinational	U_MUX (next_pc_mux)
ROM آنسکرون ۳۲×۱۶ بیت	Combinational	U_ROM (instr_mem)
رجیستر ۳۲ بیتی با flush و freeze	Sequential	U_IFID (if_id_reg)
محاسبه PC+۱	Combinational	جمع‌کننده
بافرهای ورودی/خروجی FPGA	IO Buffer	IBUF/OBUF

وجود IBUF برای ورودی‌ها و OBUF برای خروجی‌ها نشان‌دهنده نگاشت صحیح سیگنال‌ها به پایه‌های فیزیکی FPGA است. سیگنال freeze به پایه E (Enable) رجیسترها متصل شده که تأیید می‌کند منطق توقف Pipeline درست پیاده‌سازی شده است.

جمع‌کننده PC+1 به صورت یک بلاک کاملاً مجزا در Schematic ظاهر نمی‌شود، بلکه توسط بهینه‌ساز Vivado درون LUT های مربوط به U_MUX (شش LUT از نوع LUT1 تا LUT6) ادغام شده است. این رفتار طبیعی برای جمع‌کننده‌های کوچک ترکیبی در فرآیند سنتز است و تأثیری بر صحت عملکردی مدار ندارد.

۷. تحلیل Resource Utilization



شکل ۱۰ - گزارش Resource Utilization پس از سنتز

تحلیل منابع:

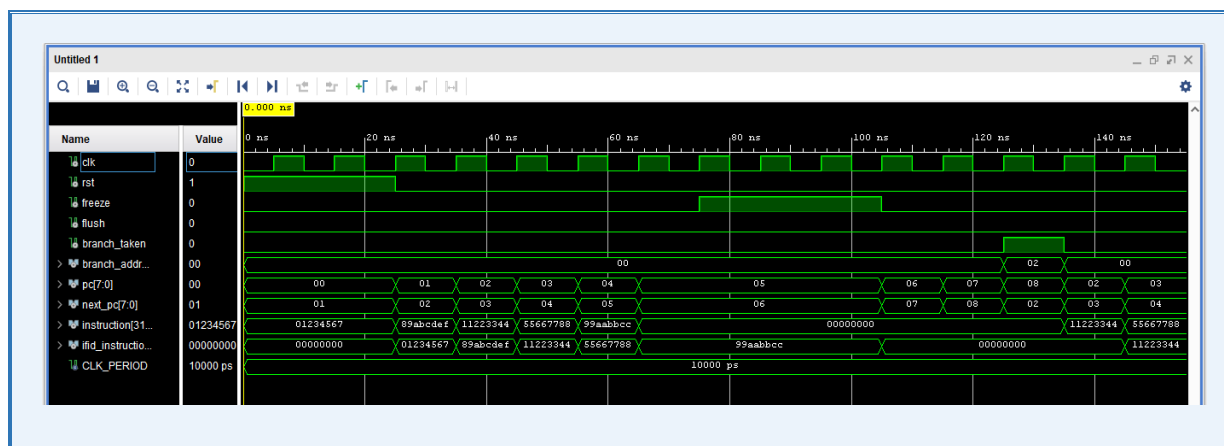
- LUT (Look-Up Table): ۱۹ عدد برای پیاده‌سازی منطق ترکیبی (ROM، MUX، جمع‌کننده)
- FF (Flip-Flop): ۱۶ عدد برای رجیسترهای ترتیبی (۸ تا برای PC + ۸ برای IF/ID)
- IO: ۹۳ پایه از ۱۰۶ پایه موجود (۸۷.۷۴٪) به دلیل وجود خروجی‌های ۳۲ بیتی instruction و ifid_instruction

میزان استفاده از LUT و FF بسیار کم است (زیر ۰.۱٪) که نشان‌دهنده سادگی و بهینه بودن طراحی است. تنها IO به دلیل تعداد زیاد پایه‌های خروجی ۳۲ بیتی قابل توجه است.

۸. تحلیل Waveform شبیه‌سازی

در Waveform سیگنال‌های زیر بررسی شدند: clk, rst, freeze, flush, branch_taken, branch_address, pc, next_pc, instruction, ifid instruction

بخش اول: Reset (۰ تا ۲۵ ns)



شکل ۱۱ - Waveform؛ / ابتدا تا ۱۴۰ ns: Reset، اجرای عادی، Freeze و Branch

تحليل:

- هنگام $rst=1$: $PC=0x00$, $instruction=0x01234567$ (محتوای خانه ۰ ROM),
 $ifid_instruction=0x00000000$
- پس از $rst=0$: PC شروع به شمارش می‌کند: $0.4 < 0.3 < 0.2 < 0.1 < 0.0$
- $instruction$ به‌صورت ترکیبی تغییر می‌کند: $0x1234567 < 89abcdef < 11223344 < 55667788$
 $99aabbcc$
- $ifid_instruction$ دقیقاً یک سیکل کلاک عقب‌تر از $instruction$ است (تأخیر رجیستر IF/ID)

بخش دوم: Freeze (۷۵ تا ۱۰۵ ns)

تحليل:

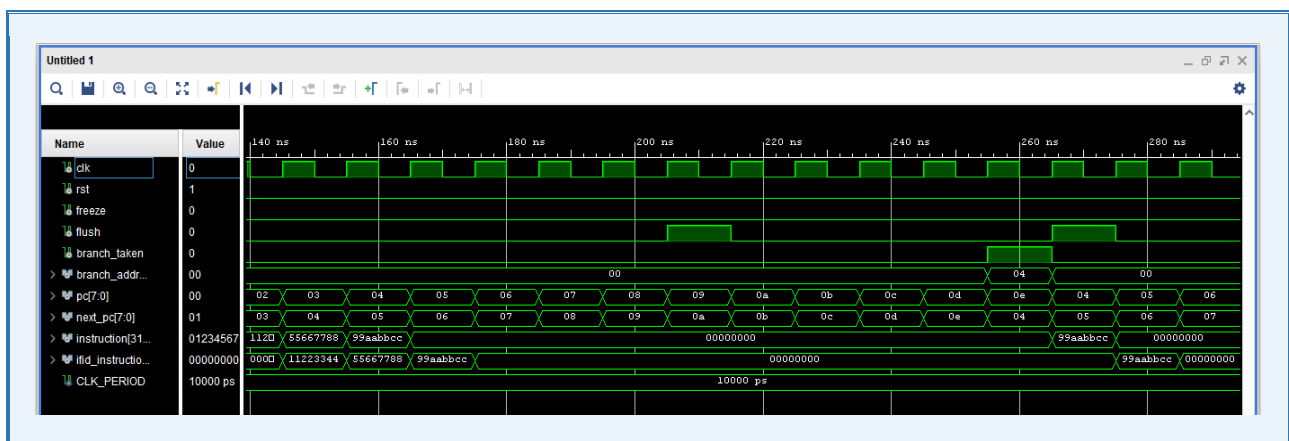
- هنگام $freeze=1$: PC ثابت می ماند (در مقدار 0×05)
- `ifid_instruction` هم تغییر نمی کند (دستور قبلی حفظ می شود)
- `instruction` نیز ثابت می ماند چون PC تغییر نکرده
- این رفتار دقیقاً همان Stall است که برای رفع Data Hazard استفاده می شود

بخش سوم: Branch (۱۲۵ تا ۱۳۵ ns)

تحلیل:

- هنگام $branch_taken=1$ و $branch_address=0x02$: $next_pc=0x02$
- در لبه بعدی کلاک: $PC \leftarrow 0x02$ (پرش از آدرس جاری، مثلاً $0x08$ ، به عقب به آدرس $0x02$)
- PC از $0x02$ به بعد ادامه می‌دهد: $0x02 < 0x03 < 0x04 < 0x05 \dots$
- instruction به مقدار خانه ۲ ROM برمی‌گردد ($0x11223344$)
- انتخاب عمدی آدرس $0x02$ (به جای آدرسی نزدیک PC جاری) باعث شد پرش در Waveform کاملاً محسوس و غیرقابل اشتباه باشد

بخش چهارم: Flush (۲۰۵ تا ۲۱۵ ns)



شکل ۱۲ - Waveform از ۱۴۰ تا ۲۸۰ ns عملکرد Flush

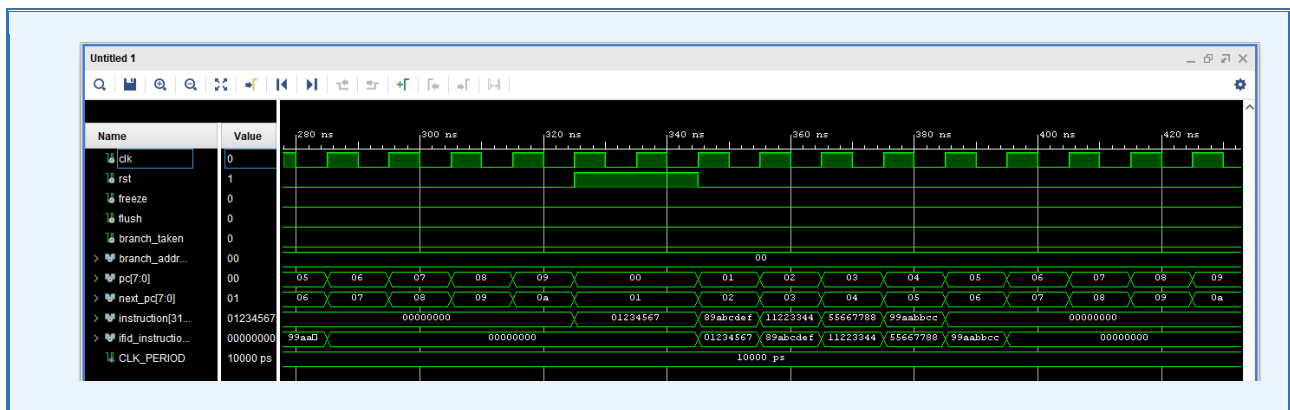
تحلیل:

- پیش از Flush، مقدار معنادار $0x11223344$ را نگه داشته بود
- هنگام $flush=1$ به مدت یک سیکل: $ifid_instruction \leftarrow 0x00000000$ (NOP)
- این دستور اشتباه که پس از Branch وارد Pipeline شده بود حذف شد؛ تغییر مقدار از $0x11223344$ به $0x00000000$ در Waveform کاملاً قابل مشاهده است
- در سیکل بعد $flush=0$ و Pipeline به حالت عادی بازمی‌گردد

بخش پنجم: Branch + Flush ترکیبی (۲۵۵ تا ۲۷۵ ns):

- branch_address=0x04 و branch_taken=1 اعمال شد؛ PC از آدرس جاری (0x0e) به 0x04 پرید
- بلافاصله flush=1 اعمال شد تا دستوری که در همان سیکل به‌اشتباه در IF/ID بارگذاری شده بود پاک شود
- این سناریو دقیقاً رفتار واقعی یک پردازنده Pipeline پس از تشخیص Branch را شبیه‌سازی می‌کند: تغییر مسیر PC همراه با حذف دستورات نادرست از مرحله بعدی

بخش ششم: Reset (۳۲۵ تا ۳۴۵ ns):



شکل ۱۳ - Waveform از ۲۸۰ تا ۴۲۰ ns عملکرد Reset

- در انتهای stimulus، rst=۱ دوباره اعمال شد
- PC و IF/ID هر دو بلافاصله به 0x00 و NOP بازگشتند
- این نشان می‌دهد رفتار Reset حتی پس از یک اجرای طولانی و چند سناریوی پیچیده (Branch، Freeze، Flush) همچنان صحیح و قابل اعتماد است

بخش هفتم: ادامه اجرا (۴۲۰ تا ۱۰۰۰ ns):



شکل ۱۴ - Waveform از ۴۲۰ تا ۱۰۰۰ ns

- پس از پایان تمام سناریوهای تعریف‌شده در stimulus process (Flush, Branch, Freeze, Reset) فرآیند stimulus با دستور wait متوقف می‌شود، اما clock process همچنان به‌صورت بی‌نهایت (while true loop) ادامه می‌یابد
- در این بازه rst=0، freeze=0، flush=0 و branch_taken=0 هستند؛ بنابراین پردازنده در حالت واکنشی ترتیبی عادی (Normal Sequential Fetch) باقی می‌ماند و PC بدون وقفه افزایش می‌یابد

- چون آدرس ورودی ROM تنها ۴ بیت کم‌ارزش PC است (pc_s(3 downto 0))، در حالی که PC خودش ۸ بیتی است، آدرس‌دهی ROM هر ۱۶ سیکل (۱۶۰ ns) تکرار می‌شود؛ در نتیجه الگوی instruction به‌صورت دوره‌ای تکرار می‌گردد: $0 \rightarrow 0x55667788 \rightarrow 0x11223344 \rightarrow 0x89abcdef \rightarrow 0x01234567 \rightarrow 0x99aabbcc \rightarrow 0x00000000$ (۱۱ بار تا تکمیل دوره ۱۶ تایی)
- این رفتار wrap-around به‌درستی و بدون هیچ Glitch یا مقدار نامعتبر در طول اجرای طولانی‌مدت رخ می‌دهد
- ifid_instruction در تمام این بازه دقیقاً یک سیکل کلاک نسبت به instruction تأخیر دارد، که نشان‌دهنده پایداری عملکرد رجیستر IF/ID در اجرای طولانی است
- این بخش عملاً رفتار Steady-State پردازنده را در غیاب هرگونه رویداد Hazard نشان می‌دهد و تأیید می‌کند طراحی برای اجرای پیوسته و طولانی‌مدت پایدار است

۹. پاسخ به سوالات تحلیل نتایج

سوال ۱: چرا مرحله IF ترکیبی از مدارات ترتیبی و ترکیبی است؟

واحد IF هم مدارات ترتیبی و هم ترکیبی دارد:

- مدارات ترتیبی: PC (رجیستر ۸ بیتی) و رجیستر IF/ID (رجیستر ۳۲ بیتی) - این‌ها وضعیت فعلی Pipeline را نگهداری می‌کنند
 - مدارات ترکیبی: ROM آسنکرون، مالتی‌پلکسر Branch و جمع‌کننده PC+۱ - این‌ها بلافاصله پاسخ می‌دهند
- دلیل این ترکیب: رجیسترها برای هماهنگی بین سیکل‌های کلاک لازمند، اما محاسبات سریع (آدرس بعدی، دسترسی به ROM) باید در همان سیکل انجام شوند تا تأخیر Pipeline کمینه شود.

سوال ۲: چرا هنگام Stall باید هم PC و هم IF/ID متوقف شوند؟

هدف Stall این است که دستور فعلی در IF Stage بدون تغییر باقی بماند:

- اگر فقط PC متوقف شود: PC ثابت می‌ماند ولی IF/ID در سیکل بعد دستور جدید (همان دستور جاری) را بارگذاری می‌کند. این باعث پیشرفت یک‌طرفه می‌شود.
- اگر هر دو متوقف شوند: PC ثابت می‌ماند پس instruction ثابت است؛ IF/ID هم ثابت می‌ماند پس دستور اشتباهی به مرحله ID نمی‌رسد. Pipeline واقعاً متوقف می‌شود.

سوال ۳: اگر فقط PC متوقف شود چه اتفاقی رخ خواهد داد؟

اگر فقط PC متوقف شود ولی IF/ID آزاد باشد:

- PC ثابت می‌ماند -> instruction ثابت می‌ماند
- اما IF/ID در هر سیکل همان instruction را بارگذاری می‌کند
- این دستور (که Stall شده) هر سیکل وارد مرحله ID می‌شود
- نتیجه: دستور به‌اشتباه دو بار اجرا می‌شود -> خطای منطقی در برنامه

سوال ۴: نقش رجیستر IF/ID در عملکرد صحیح Pipeline چیست؟

- جداسازی مراحل: رجیستر IF/ID مرز بین مرحله IF و ID است. اطلاعات را یک سیکل نگه می‌دارد تا مرحله ID بتواند در همان سیکل روی یک دستور کار کند که IF روی دستور بعدی کار می‌کند.
- پشتیبانی از Freeze: با ثابت نگه داشتن مقدار هنگام Stall، مرحله ID همان دستور را می‌بیند
- پشتیبانی از Flush: با بارگذاری NOP، دستور اشتباه (پس از Branch) قبل از رسیدن به ID حذف می‌شود
- هماهنگی زمانی: بدون این رجیستر، تغییر instruction در IF مستقیماً به ID می‌رسید و همگام‌سازی بین مراحل مختل می‌شد

سوال ۵: تفاوت Flush و Freeze چیست؟

Freeze (Stall)	Flush	
توقف Pipeline	پاک‌سازی دستور اشتباه	هدف
Structural Hazard یا Data Hazard	Control Hazard (Branch)	علت
ثابت می‌ماند	ادامه پیدا می‌کند	تأثیر بر PC
مقدار قبلی را حفظ می‌کند	NOP بارگذاری می‌شود	تأثیر IF/ID
چند سیکل ممکن است	معمولاً یک سیکل	مدت

سوال ۶: چرا پس از وقوع Branch نیاز به Flush داریم؟

در یک Pipeline بدون Branch Prediction:

۱. در سیکل N دستور Branch در مرحله EX است و نتیجه (branch_taken) مشخص می‌شود
۲. در همین سیکل، مراحل IF و ID دستورات بعد از Branch را اشتباهاً واکنشی کرده‌اند
۳. این دستورات اشتباه باید قبل از رسیدن به مرحله WB پاک شوند
۴. Flush با قرار دادن NOP در رجیستر IF/ID این دستورات را خنثی می‌کند

بدون Flush، دستورات پس از Branch (که نباید اجرا شوند) به مراحل ID، EX و MEM می‌رسند و داده‌های اشتباه می‌نویسند.

سوال ۷: چرا در این آزمایش از $PC+1$ و نه $PC+4$ استفاده شده است؟

در این آزمایش:

- هر خانه حافظه دقیقاً یک دستور کامل ۳۲ بیتی است
 - حافظه Word-Addressable است (نه Byte-Addressable)
 - آدرس ۰ اولین دستور، آدرس ۱ دومین دستور و... است
 - بنابراین برای رفتن به دستور بعدی فقط باید PC را یک واحد افزایش داد: $PC+1$
- در MIPS واقعی:

- حافظه Byte-Addressable است
- هر دستور ۴ بایت فضا اشغال می‌کند
- پس برای رفتن به دستور بعدی باید ۴ آدرس بایت رد کرد: $PC+4$

سوال ۸: اگر ROM به صورت Synchronous طراحی شود چه تغییری در Timing رخ می‌دهد؟

ROM فعلی آنسکرون است: به محض تغییر PC، instruction فوری تغییر می‌کند (در همان سیکل).

اگر ROM سنکرون باشد:

- خروجی ROM فقط در لبه بالارونده کلاک به‌روزرسانی می‌شود
 - یعنی یک سیکل تأخیر اضافه می‌شود: $PC < -$ (تأخیر) $< -$ instruction $< -$ IF/ID
 - رجیستر IF/ID باید یک سیکل دیرتر مقدار را لچ کند
 - این باعث می‌شود Pipeline نیاز به یک سیکل اضافه در مرحله IF داشته باشد
 - نتیجه: کاهش کارایی (افزایش CPI) یا نیاز به تغییر در طراحی Testbench
- مزیت ROM سنکرون: سازگاری بهتر با BRAM (Block RAM) در FPGA که سرعت بالاتری دارند.

سوال ۹: استفاده از NOP چه تأثیری بر CPI پردازنده دارد؟

CPI (Cycles Per Instruction) معیار کارایی Pipeline است.

- در حالت ایده‌آل: $CPI = 1$ (یک دستور در هر سیکل تکمیل می‌شود)
- هر بار NOP وارد Pipeline شود، یک سیکل کلاک هدر می‌رود بدون اینکه کاری مفید انجام شود

- هر Flush یک NOP تولید می‌کند: $CPI \approx 1$ به ازای هر Branch
 - اگر Branch Prediction نداشته باشیم و Pipeline عمق ۵ داشته باشد: حتی ۲-۳ سیکل کارایی از دست می‌رود
- راه‌حل‌های کاهش تأثیر NOP:
- Branch Prediction (پیش‌بینی شاخه)
 - Delayed Branch (اجرای یک دستور بعد از Branch)
 - Branch Target Buffer (BTB)

۱۰. نتیجه‌گیری

در این آزمایش واحد (IF Stage) Instruction Fetch یک پردازنده Pipeline با موفقیت طراحی، پیاده‌سازی و ارزیابی شد. چهار ماژول مجزا (pc_reg، next_pc_mux، instr_mem، if_id_reg) طراحی و در یک Top Module به هم متصل شدند.

نتایج شبیه‌سازی نشان داد:

- اجرای عادی: PC به‌درستی از 0×00 شمارش می‌کند و instruction متناظر را از ROM واگشی می‌کند
- Freeze: هم PC و هم IF/ID با موفقیت ثابت ماندند (Stall صحیح پیاده‌سازی شد)
- Branch : PC با موفقیت به آدرس 0×02 پرید و شمارش را از آنجا ادامه داد؛ همچنین سناریوی ترکیبی Branch+Flush به‌درستی رفتار واقعی Pipeline پس از یک شاخه را شبیه‌سازی کرد
- Flush: رجیستر IF/ID با موفقیت NOP بارگذاری کرد (دستور اشتباه پاک شد)
- تأخیر IF/ID : ifid_instruction دقیقاً یک سیکل عقب‌تر از instruction بود

نتایج سنتز نیز نشان داد:

- مدار بدون خطا سنتز شد و قابلیت پیاده‌سازی روی ۱-xcy7a5tcbg۲۳۶ FPGA را دارد
- منابع استفاده شده بسیار کم است: ۱۹ LUT ، ۱۶ FF (زیر ۰.۱٪)
- RTL Schematic ساختار صحیح همه ماژول‌ها و ارتباطات بین آنها را تأیید کرد

در نتیجه تمام اهداف آزمایش شامل درک مفهوم Pipeline، طراحی واحد IF، پیاده‌سازی Freeze و Flush، تحلیل Waveform و بررسی سخت‌افزاری با موفقیت محقق شدند.